

# Generative Adversarial Networks (GANs)

---

A **Generative Adversarial Network (GAN)** is a deep learning framework where two neural networks—a generator and a discriminator—compete in a zero-sum game. The generator creates synthetic data while the discriminator evaluates its authenticity, with both networks improving adversarially until the generator produces highly realistic samples.

## 1. Generator ( $G$ )

- **Input:** Random noise vector  $z \sim p_z(z)$
- **Output:** Synthetic data samples  $G(z)$
- **Goal:** Learn distribution  $p_g$  that matches real data distribution  $p_{data}$

## 2. Discriminator ( $D$ )

- **Input:** Real data  $x \sim p_{data}(x)$  or fake data  $G(z)$
- **Output:** Probability  $D(x) \in [0, 1]$  of input being real
- **Goal:** Distinguish between real and generated samples

## Analogy

---

### Generator = Counterfeiter

- Creates fake currency
- Learns from police detection feedback
- Goal: Produce undetectable counterfeits

### Discriminator = Police

- Examines currency authenticity
- Studies real and counterfeit money
- Goal: Perfect detection accuracy

**Training:** Continuous improvement cycle where both adversaries enhance their skills.

## Training

---

### Training Algorithm

1. **Sample** minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$
2. **Sample** minibatch of  $m$  real examples  $\{x^{(1)}, \dots, x^{(m)}\}$
3. **Update Discriminator:** Ascend gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$$

4. **Update Generator:** Descend gradient (original):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z^{(i)})))$$

Or ascend gradient (improved):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D(G(z^{(i)})))$$

## Mathematics

---

### Objective Function

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

### Optimal Solutions

At convergence:

- Discriminator:  $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$
- Generator:  $p_g = p_{data}$
- Equilibrium:  $D^*(x) = \frac{1}{2}$  for all  $x$

## Pseudo Code

---

```

for number_of_training_epochs do
  for k_steps do
    # Train Discriminator
    Sample minibatch of m noise samples  $\{z^1, \dots, z^m\} \sim p_z(z)$ 
    Sample minibatch of m real samples  $\{x^1, \dots, x^m\} \sim p_{data}(x)$ 
    Update D by ascending:  $\nabla \theta_d [\log D(x^i) + \log(1 - D(G(z^i)))]$ 
  end for

  # Train Generator
  Sample minibatch of m noise samples  $\{z^1, \dots, z^m\} \sim p_z(z)$ 
  Update G by descending:  $\nabla \theta_g \log(1 - D(G(z^i)))$ 
end for

```

## Example:

---

### Training Progression:

- **Epoch 1-10:** Generator produces random noise
- **Epoch 11-50:** Blurry digit-like shapes emerge
- **Epoch 51-100:** Clear but imperfect digits
- **Epoch 100+:** Realistic MNIST digits

**Final Result:** Generator can produce any digit (0-9) that is visually indistinguishable from real MNIST samples.

---

## Challenges

---

## 1. Mode Collapse

**Problem:** Generator produces limited variety (e.g., only generates digit “1”)

**Cause:** Generator finds “safe” modes that fool discriminator

## 2. Training Instability

**Problem:** Oscillations without convergence

**Equation:** Non-convex game with Nash equilibrium difficulties

## 3. Vanishing Gradients

**Problem:** When  $D(G(z)) \rightarrow 0$ , generator gradients vanish:

$$\nabla \log(1 - D(G(z))) \rightarrow 0$$

## 4. Evaluation Difficulty

**Problem:** No explicit likelihood measure

**Solution:** Use Inception Score, FID, or human evaluation

---

## GANs vs Autoencoders Comparison

---

| Aspect            | GANs                      | Autoencoders                      |
|-------------------|---------------------------|-----------------------------------|
| Primary Objective | Generate new data samples | Reconstruct input data            |
| Architecture      | Generator + Discriminator | Encoder + Decoder                 |
| Training Method   | Adversarial game          | Reconstruction error minimization |
| Loss Function     | $\min_G \max_D V(D, G)$   | $L(x, g(f(x)))$                   |

| Aspect                 | GANs                            | Autoencoders                              |
|------------------------|---------------------------------|-------------------------------------------|
| Output                 | Synthetic samples               | Reconstructed input                       |
| Latent Space           | Random noise input              | Learned compressed representation         |
| Sample Quality         | <b>High</b> (sharp images)      | Lower (often blurry)                      |
| Training Stability     | Unstable                        | Stable                                    |
| Theoretical Foundation | Game theory                     | Optimization theory                       |
| Mode Coverage          | Can suffer from mode collapse   | Better data coverage                      |
| Applications           | Data generation, style transfer | Denoising, compression, anomaly detection |
| Evaluation Metrics     | IS, FID, human judgment         | Reconstruction error, MSE                 |
| Mathematical Goal      | $p_g = p_{data}$                | $g(f(x)) \approx x$                       |